Shipgen Platform

FleetOps · YMS · PMS

Operations Engines — Simple Guide

Problems · Solutions · Modules · Scope · Future AI

Version 1.0 | July 2026 | Audience: Business & Operations Teams

Executive Summary

Shipgen is one platform with three specialized engines for logistics operations. Each engine solves a different part of the supply chain — from last-mile delivery, to truck yard orchestration, to parking facility management.

Engine	Route	What It Does
FleetOps	/fleet-ops/*	Orders, drivers, vehicles, routes, dispatch & tracking
YMS (YARD.OS)	/yard/*	Truck lifecycle: appointment → gate → dock → exit
PMS (ParkFlow)	/parking/*	Parking tickets, payments, occupancy & reports

One login, role-based access. Staff see only the engine they need. Admins can switch between engines from the Shipgen console header. All three engines share IAM (identity) but run on their own services behind a single API gateway.

Why These Three Engines Matter Together

- **FleetOps** moves goods on the road — orders, drivers, and proof of delivery.
- **YMS** manages trucks when they arrive at a warehouse or hub yard.
- **PMS** manages vehicles parked at a facility — entry, payment, and exit.

Future vision: A delivery order in FleetOps could trigger a yard appointment in YMS, and parking revenue could flow to Ledger — cross-engine workflows are planned but not fully built yet.

Platform Architecture (Simple View)

All three engines follow the same pattern: **React UI embedded in Shipgen console** → **nginx gateway** → **dedicated backend service** → **database**.

Layer	FleetOps	YMS	PMS
UI Path	/fleet-ops/*	/yard/*	/parking/*
API Path	/int/v1/*	/api/yms/*	/api/pms/*
Backend	Laravel (fleetops-service)	FastAPI (yms-service)	FastAPI (pms-service)
Database	MySQL (fleetbase)	PostgreSQL	PostgreSQL (parkflow)
Mobile	Driver app (Navigator)	Yard floor app	Planned (not in repo)

Shared Platform Services

- **IAM** — users, roles, and permissions for all engines.
- **Console shell** — sidebar, header, notifications, engine switcher.
- **Gateway** — single entry point (port 8000) routes traffic to the right service.
- **SSO bridge** — Shipgen admin logs in once; Yard and Parking get their own JWT automatically.

Engine Comparison at a Glance

Criteria	FleetOps	YMS	PMS
Primary users	Dispatchers, fleet managers	Gate ops, dock supervisors	Parking operators
Core workflow	Create → Dispatch → Deliver	Arrive → Queue → Load → Exit	Enter → Pay → Exit
AI today	Rule-based warnings only	Rule-based recommendations	License plate OCR only
Maturity	High (mature backend)	Demo-ready, hardening needed	Core flows ready, UI gaps

Key takeaway: FleetOps is the largest and most mature engine. YMS and PMS are newer microservices embedded into the same Shipgen console, each with its own database and permission model.

FleetOps — Overview & Modules

FleetOps is the last-mile logistics and fleet management engine. It helps teams plan, dispatch, track, and complete delivery or service orders using drivers, vehicles, routes, and configurable workflows.

Main Module Groups

Module Group	Key Features	Route Prefix
Operations	Orders (list/kanban/map), routes, schedule, orchestrator, service rates	/fleet-ops/operations
Management	Drivers, vehicles, fleets, places, vendors, contacts, issues, fuel	/fleet-ops/management
Connectivity	Telematics, devices, sensors, fleet tracking, vehicle devices	/fleet-ops/connectivity
Maintenance	Work orders, schedules, service records, parts, equipment	/fleet-ops/maintenance
Admin / Logistics	Manifests, payloads, proofs, purchase rates, tracking numbers	/fleet-ops/admin
Geo & Analytics	Service areas, geofences, reports, custom fields, settings	/fleet-ops/geo, /analytics

Order Lifecycle (Simple)

Created → **Dispatched** (driver/vehicle assigned) → **In Progress** (workflow steps) → **Completed** or **Cancelled**. Proof of delivery (POD) and tracking are supported throughout.

Orchestrator — Smart Assignment

- Bulk assign drivers and vehicles to unassigned orders.
- Route optimization using greedy, driver-assignment, or VROOM engines.
- Preview before commit — see the plan, then apply it.

Operational Intelligence (Rule-Based)

- Delivery risk warnings (stalls, delays).
- Dispatcher suggestions (unassigned orders, idle drivers).
- Unified warning panel across the console.

Note: This is *not* machine learning. It uses simple business rules on live data.

Integrations

- Telematics: Flespi, Geotab, Samsara.
- Maps: Google Maps, OSRM routing.
- Third-party delivery: Lalamove.
- Mobile: Navigator driver app for field operations.

FleetOps — Problems & Solutions

FleetOps has a mature backend but the React console is still catching up to the legacy Ember UI. Below are the main gaps and what has been done to fix them.

Problem	Current Solution	Future Direction
Legacy Ember UI replaced by React	Full React rebuild with 27+ modules, drawer-first UX, shared CRUD scaffold	Enterprise dispatch, connectivity, maintenance, maps
API path inconsistencies	tryCandidates pattern tries multiple API paths	Single canonical API per resource, idempotent dispatch handling
Connectivity module shallow (~42%)	Telematics setup wizard, device panels, webhook ingest started	Full sensor/event storage, WebSocket broadcasting
Maintenance module shallow (~42%)	New forms: work orders, schedules, parts, equipment, warranties	Full E2E tests, dedicated vehicle maintenance tab
Maps & service areas (~32–40%)	Google map editor, zone forms, boundary selection added	Full geofence drawing, zone geocoding edge cases
Orchestrator UI incomplete	Preview/commit flow, engine selection, import modal built	Full VRP depth, server-side pagination at scale
Permissions not fail-closed	sidebarAccess.js ability checks per resource	Full SaaS permissions audit (~50% today)
No link to YMS appointments	Engines run independently with SSO	FleetOps order → YMS appointment sync (planned)

Priority Fixes Still Needed

- **P0:** Server-side order pagination for large fleets.
- **P0:** Permissions fail-closed for multi-tenant SaaS.
- **P1:** Telematics webhook event/sensor storage completion.
- **P1:** Extension registry virtual tabs (~10% coverage).
- **P2:** Dedicated fleetops_db schema split (microservices Phase 2).

Overall coverage: ~65% domain average. Dispatch workflows are ~82–85% complete. Source: FLEETOPS-GAPS.md internal audit.

YMS (YARD.OS) — Overview & Modules

YMS (YARD.OS) is the Smart Yard Management System. It digitizes the full truck lifecycle at a warehouse or logistics hub — from scheduled appointment through gate check-in, virtual queue, dock assignment, loading, and exit.

Vehicle Lifecycle (15 Statuses)

DRAFT → SCHEDULED → ARRIVED → CHECKED_IN → WAITING → CALLED → DOCK_ASSIGNED → RESOURCE_PENDING → READY_FOR_LOADING → LOADING → COMPLETED → EXIT_HOLDING → EXIT_VERIFIED → EXITED (or CANCELLED). Each transition is enforced by the backend.

Main Modules

Module	Route	Purpose
Control Tower	/yard	Live KPIs, alerts, queue/dock snapshots
Recommendations	/yard/ai	Rule-based guidance from live alerts
Virtual Queue	/yard/queue	Priority scoring, call-in, supervisor override
Gate	/yard/gate	Entry verification, exit holding, gate-out
Docks	/yard/docks	Bay assignment, release, utilization
Loading Ops	/yard/loading	Active loading, exceptions, progress
Yard Map	/yard/yard	Zone occupancy view (derived, not GIS)
Appointments	/yard/appointments	Slot booking, calendar, exports
Detention	/yard/detention	Wait-time billing, approve/dispute/paid
Resources	/yard/vehicles, /equipment, /drivers	Registry and assignment
Reports	/yard/reports/*, /kpis	Delay, SLA, dock utilization, executive KPIs
Admin	/yard/admin/*, /settings	Users, roles, facility settings

Control Tower — 9 Alert Types

Vehicle waiting too long · Hazmat SLA breach · Loading delay · Loading exception · Labor unavailable · Equipment unavailable · Dock blocked · Exit holding delay · Queue congestion. All computed server-side from live yard data.

Roles (5 Production Roles)

Yard Administrator · Yard Manager · Gate Operator · Yard Coordinator · Dock Supervisor. Each role sees different modules in the sidebar and can only perform allowed actions.

YMS — Problems & Solutions

YMS core workflows are production-ready for demo and pilot use. Security hardening and enterprise features are the main remaining work.

Problem	Current Solution	Future Direction
Fragmented yard state on paper/radio	Single digital lifecycle with enforced status transitions	Multi-facility tenancy
No real-time visibility for managers	Control Tower with 30s refresh + cross-module navigation	WebSocket/SSE instead of polling
Detention revenue leakage	Detention module with formula, approve/dispute/paid workflow	ERP export (SAP/Oracle/Tally)
Dock under-utilization	Dock assignment, utilization reports, queue visibility	ML-based dock prediction
Slow gate throughput	Gate lookup, approve/reject, verify workflows	OCR/ANPR gate automation (stub exists)
GET APIs bypass RBAC (Critical)	Write APIs fully guarded; frontend route guards active	Enforce RBAC on all GET endpoints
Admin UI is demo-only (localStorage)	Backend ROLE_PERMISSIONS is source of truth	Backend-backed user/role admin API
No FleetOps integration	SSO bridge for Shipgen admins; standalone YMS available	FleetOps appointment auto-sync

Production Readiness Scores

Area	Score	Status
Core workflows	Production-ready	Gate, queue, docks, loading tested
Overall production	62/100	Hardening needed
Security	68/100	GET RBAC gap is critical
RBAC	78/100	Write guards good; read guards weak
Backend tests	122+ pytest	Workflow coverage solid

“AI” in YMS is the Recommendations page (/yard/ai). It maps Control Tower alerts to suggested actions. The UI clearly states: *“Rule-based recommendations — not predictive AI.”*

PMS (ParkFlow) — Overview & Modules

PMS (ParkFlow) is the Parking Management System. It digitizes parking facility operations — replacing paper tickets with QR codes, tracking occupancy by floor and vehicle category, collecting payments, and giving supervisors live monitoring and revenue reports.

Three Staff Roles

Role	Main Responsibilities	Example Routes
Admin	Employees, pricing, floors, hardware, reports, audit	/parking/admin/*
Supervisor	Live monitoring, operator activity, QR scans, alerts	/parking/supervisor/*
Operator	New ticket, collect payment, vehicle search, exit, QR print	/parking/operator/*

Ticket Lifecycle (Simple)

Created (paid or unpaid at entry) → **Paid** → **Exited**. Entry flow: plate number (manual or OCR) → vehicle category → pricing → floor assignment → QR ticket. Exit flow: search vehicle → verify payment → free slot → decrement occupancy.

Main Modules

Module	Purpose
Dashboard	Role-specific KPIs (entries, revenue, occupancy)
New Ticket / OCR	Create ticket; auto-fill plate from photo (EasyOCR)
Collect Payment	Search unpaid ticket → collect → audit log
Vehicle Search / Exit	Find vehicle → process exit → update occupancy
Floor Management	Multi-floor capacity by category (2W / 4W / heavy)
Pricing Rules	Base price, hourly/daily rates (stored)
Monitoring	Live entry/exit feed, operator activity, QR scan events
Reports	Revenue, traffic, category breakdown + PDF export
Hardware Registry	Device registry and status (no automation yet)
Audit Logs	Full trail of login, ticket, payment, exit actions

Vehicle Categories & Floors

Supports 2-wheeler, 4-wheeler, and heavy vehicles. Floors have per-category capacity limits. System auto-assigns the best available floor or returns a capacity error.

PMS — Problems & Solutions

PMS core ticketing, occupancy, and reporting work well. Gaps are mainly in settings UI, pricing depth, payment enforcement, and hardware automation.

Problem	Current Solution	Future Direction
Paper tickets, no digital tracking	QR-coded digital tickets with full audit trail	ANPR camera direct integration
No occupancy visibility	Real-time floor occupancy by category with auto-decrement	Multi-facility assignment
Manual plate entry errors	EasyOCR license plate recognition on photo upload	Live camera ANPR at gate
No supervisor oversight	Monitoring dashboard, operator activity, QR scan events	Real-time notification/alert system (bell is decorative today)
Separate login for parking staff	SSO bridge for Shipgen admins; parking-only accounts (parking_role)	IAM packing (parking_role) for non-admin Shipgen users
Hourly/daily pricing not calculated	Pricing rules stored; base_price applied at ticket creation	Full pricing engine at ticket time
Unpaid vehicles can exit	Backend supports pay_now flag; unpaid-at-exit possible	Block exit until payment collected
Settings forms not wired	UI forms exist for facility settings	Connect settings forms to backend API
No boom barrier control	Hardware device registry with status tracking	Automatic gate/boom barrier integration
Revenue not in accounting	Reports with PDF export in PMS	Ledger invoice sync (cross-engine, planned)

E2E Test Status

102 Playwright tests: 51 passed, 51 skipped. Core operator and admin flows are covered. Skipped tests are mostly for unwired settings and advanced scenarios.

Future AI Features & Cross-Engine Vision

Today, **none of the three engines use generative AI or machine learning** for core operations. What exists is rule-based logic and one OCR feature. Below is the current state and future roadmap.

AI & Intelligence — Current vs Future

Engine	What Exists Today	Future AI / ML Plans
FleetOps	Rule-based delivery risks, dispatcher suggestions, orchestrator engines (greedy/VROOM)	Predictive ETA, demand forecasting, auto-dispatch ML, route learnin
YMS	9 Control Tower alert rules; Recommendations page maps alerts to actions	ML delay prediction, dock optimization, ANPR gate automation, capa
PMS	EasyOCR license plate recognition from uploaded photos	Live ANPR cameras, occupancy prediction, dynamic pricing AI

Cross-Engine Integration (Planned)

- **FleetOps → YMS:** Delivery order triggers yard appointment automatically.
- **YMS → FleetOps:** Truck exit updates order status; telematics auto-detects ARRIVED.
- **PMS → Ledger:** Parking revenue syncs to invoices and accounting.
- **All engines → IAM:** Fine-grained role mapping across engines (in progress).
- **Mobile:** Driver app (FleetOps), yard floor app (YMS), parking app (PMS — planned).

Platform-Wide Future Enhancements

Priority	Enhancement	Engines Affected
High	Enterprise SSO (OAuth2/SAML)	All three
High	GET-level RBAC enforcement	YMS, PMS
High	Multi-facility / multi-tenant support	YMS, PMS
Medium	WebSocket real-time updates (replace 30s polling)	YMS, FleetOps
Medium	ERP/WMS/TMS connectors	YMS, FleetOps
Medium	Dedicated database per service (schema split)	FleetOps
Low	Predictive analytics dashboards	All three
Low	GIS-based yard map with zone master	YMS

BRD policy: Predictive AI is currently *out of scope* for FleetOps. Recommendations are rule-based only. Any future ML features will be additive, not replacements for enforced workflows.

Scope Summary & Roadmap

What Is In Scope (These Three Engines)

- **FleetOps:** Last-mile orders, fleet assets, routes, dispatch, telematics, maintenance, orchestrator, mobile driver app.
- **YMS:** Full truck yard lifecycle, gate/queue/dock/loading, detention billing, resources, reports, mobile yard app.
- **PMS:** Multi-floor parking, ticketing, payments, occupancy, monitoring, reports, plate OCR.
- **Shared:** Shipgen console embedding, IAM auth, API gateway, role-based sidebar, SSO bridges.

What Is Out of Scope (Today)

- Advanced TMS with full OR-Tools / commercial solver (basic routes only).
- Generative AI / LLM assistants in any engine.
- Automatic boom barriers or ANPR cameras (monitoring/registry only).
- Multi-facility enterprise tenancy for YMS and PMS.
- Containers, customs, ports, freight forwarding (YMS deferred).
- Ledger auto-sync from parking revenue.

12-Month Roadmap (Suggested Priority)

Quarter	FleetOps	YMS	PMS
Q1	Pagination + permissions hardening	GET RBAC + real admin API	Wire settings + exit payment gate
Q2	Connectivity & maintenance depth	OCR gate automation pilot	Full pricing engine + Ledger sync
Q3	FleetOps → YMS appointment sync	ERP connector + multi-facility	ANPR camera integration
Q4	Predictive ETA pilot (ML)	ML delay prediction pilot	Dynamic pricing pilot

Key Documentation References

- docs/SHIPGEN-BRD.md — full business requirements
- frontend/docs/FLEETOPS-GAPS.md — FleetOps gap analysis
- yms/docs/YMS_Project_Overview.md — YMS project overview
- docs/PARKING-PMS-DOCUMENTATION.md — PMS full documentation
- docs/YMS-INTEGRATION.md — YMS embedding guide
- docs/MICROSERVICES-ARCHITECTURE.md — service architecture

Summary: Shipgen delivers three focused engines that together cover road logistics, yard orchestration, and parking management. Each engine has working core workflows today, documented gaps with active solutions, and a clear path toward AI-assisted operations and cross-engine integration.